Agents Read More Than They Use

Learning Tool-Output Projections from Execution Traces, and Where They Fail

Alessio Pomaro
Independent Researcher, ORCID 0009-0008-6975-996X

Preprint, tracetrim 0.3.3 (studies run with versions 0.1 and 0.2)

Abstract

Agents built on large language models read tool outputs written for programs: most of what an API returns never informs the agent's answer, yet it is paid for at every turn. We learn, from an agent's own execution traces, which fields of each tool output it uses, and remove the rest. On five agents (three on real APIs, two on simulated servers following public OpenAPI descriptions), a projection learned from seven runs and applied to an unseen one removed 63% to 77% of tool-output tokens in four cases and 12% in the fifth; in live runs with an `expand` tool for hidden fields, billed input tokens fell by 56% and 32% with every quality gate passed. Learning only from tool calls, as a gateway would, missed most of the fields agents use. Replays with projected outputs, under a protocol frozen in advance, located the failures precisely: values the agent transforms before writing them, fields it uses only to reason, and, without an identifier field, agents inventing identifiers. A second pre-registered study showed that two safeguards, keeping identifying fields and recognising transformed values, removed those losses while keeping savings of 46% to 61%; a post-hoc check found a fourth mechanism, translated values, not covered. We release the library, with a bilingual telemetry report and an MCP proxy that applies a reviewed projection in front of any server.

Status. Preprint, not peer reviewed. DOI 10.5281/zenodo.23953934.

1 Introduction

An agent built on a large language model works by calling tools and reading what they return. The tools are usually written for programs, not for models: a keyword-research API returns monthly volumes, click-through data, backlink counts and a dozen normalisations of each; a payment API returns every field of a charge, null or not; a code-hosting API embeds a full user object, with fifteen navigation URLs, in every issue. All of it enters the context, is paid for at every subsequent turn, and competes for the model's attention with what the task needs.

Practitioners handle this by hand: a wrapper function per tool that keeps the fields the task uses. This works, but it requires knowing which fields those are, for each tool and each task, and keeping the wrappers in step with the APIs. Generic compressors avoid the per-tool work but do not know what the task needs, so they remove structural redundancy and leave the semantic one.

This paper asks whether the hand-written selection can be *learned*. The agent's own execution traces show what it did with each tool output: which values reappear in the arguments of later calls and in the final answer. A field whose values never reappear, across runs of the same task, is a candidate for removal. We call the resulting filter a *learned projection*, implement it in an open-source library (`tracetrim`), and test it on five agents: three that use real APIs (an editorial planner and a content-gap analyst on a search-data API, a literature scanner on a scholarly catalogue) and two that use simulated servers whose payloads follow the public OpenAPI descriptions of a payment API and a code-hosting API.

The study has two parts. **Study 1**, with a protocol frozen before any run, measures how much a projection learned from other runs removes on a held-out run, where it must be learned

to work, whether it transfers between tasks, and whether the agent's answer survives when the conversation is replayed with projected tool outputs. It finds large savings and three precise ways in which the projection fails: values the agent *transforms* before writing them (a Unix timestamp written as a date), fields the agent uses only *to reason*, and, most seriously, an agent that *invents identifiers* when the identifier field has been removed. **Study 2** adds two safeguards motivated by those failures and tests, again with a protocol frozen in advance, whether they remove the losses while keeping most of the saving.

The contributions are: (i) a method to learn tool-output projections from traces, with a leave-one-out evaluation that estimates savings and misses on runs the projection has never seen; (ii) evidence that the learner must see the agent's final answer, so a proxy placed only between agent and tools (an MCP gateway) cannot learn by itself; (iii) a characterisation of the failure modes, including identifier fabrication; (iv) safeguards and their test; (v) an open-source implementation with a bilingual telemetry report that is useful on its own, independently of any trimming.

2 Related work

Prompt and context compression. A large body of work compresses what a language model reads. Token-level methods drop tokens scored as uninformative, by self-information [7] or by a small language model [3], possibly conditioned on the question [4]; LLMLingua-2 learns a task-agnostic compressor [13]. RECOMP compresses retrieved documents for retrieval-augmented generation [19], and gist tokens learn soft prompt representations [12]. These methods operate on text and decide token by token; the projection studied here operates on structured outputs and decides field by field, and its output is still readable, diffable JSON. The motivation is shared: models use information in long contexts unevenly [9], so what is removed is not only cost.

Tool use and the agent loop. Agents that interleave reasoning and tool calls [20], learn when to call APIs [15] or operate over thousands of real APIs [14] accumulate tool observations at every turn. The Model Context Protocol [11] has made connecting third-party tools routine, which moves the choice of what an agent reads from the agent's author to the tool's.

Compressing what agents observe. Closest to this work are methods that reduce agent observations. ACON optimises, in natural language, compression guidelines for observations and history from the agent's failures, and distils the compressor into smaller models [5]. For software-engineering agents, masking older observations matches LLM summarisation at half the cost [8]. Squeez fine-tunes a small model to return, for a focused query and one tool output, the smallest verbatim evidence block the agent should inspect next [6]. Our setting differs in three ways: the projection is learned from the agent's own traces without training or calling any model, so it adds no inference cost and its decisions are a reviewable list of fields; it targets structured outputs of arbitrary APIs rather than code-agent observations; and we measure where such learning fails, not only how much it saves.

Tools. Headroom compresses agent context in front of the model, routing content to compressors by type [2]; TOON is a compact, table-oriented serialisation of JSON for language models [16]. Both are structural: they do not decide which fields a task needs, and we compare against them as such. Gateways for the Model Context Protocol reduce context in other places: mcp-compressor compresses the descriptions and schemas of the tools exposed [1], and Sift stores full payloads outside the context and lets the agent query them [10], which is close in spirit to our `expand`.

Fabricated identifiers. Language models fabricate plausible identifiers when the real ones are not in front of them: bibliographic references [18] and software package names [17]. We observe

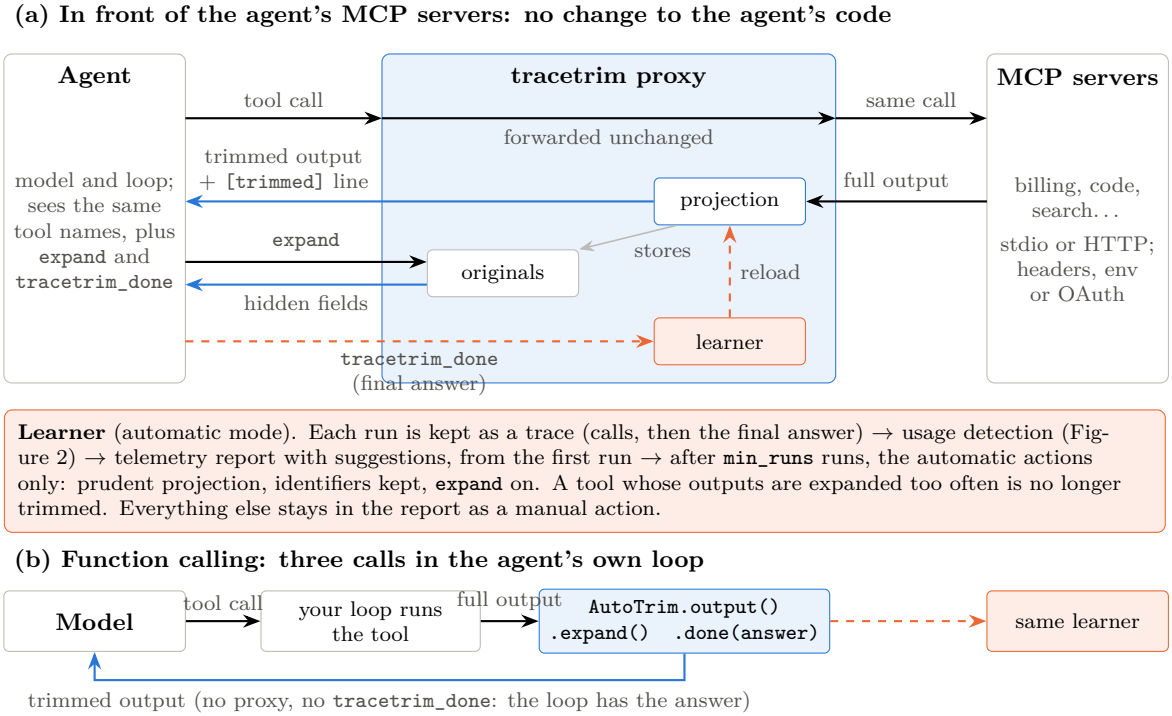


Figure 1: Where tracetrim acts. (a) As an MCP proxy, it stands in for the agent's servers: calls are forwarded unchanged, only results are rewritten on their way back, and a trimmed result ends with a line listing what was hidden, which `expand` returns from the stored original without calling the server again. The agent passes its final answer to `tracetrim_done`, which closes a run for the learner. (b) With function calling the same learner sits in the agent's own loop. The automatic mode (dashed) is part of version 0.3 and was not evaluated in the studies reported here, which used offline learning and a fixed projection.

the same behaviour inside an agent loop, caused by the removal of an identifier field, which makes identifiers a category a projection must never remove.

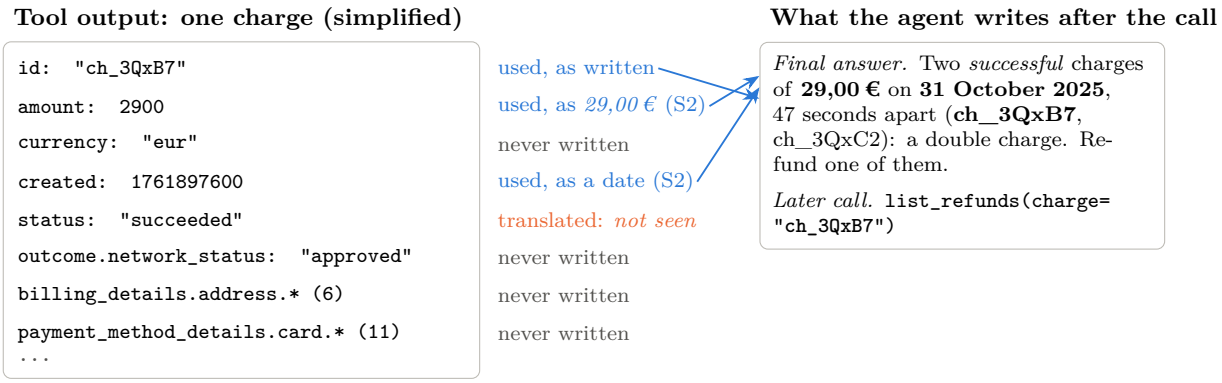
3 Method

Figure 1 shows where the library acts and how it learns; Figure 2 shows how it decides which fields an agent uses. The rest of this section describes each step.

Traces. A trace is the ordered record of what the agent was given and what it wrote: prompts, tool calls (the arguments the agent wrote and the raw output it received) and messages, the final answer included. The library reads a neutral JSONL format and the Python exports of the OpenAI Playground.

Usage. Each structured output is flattened into leaves identified by a path in which list indices are collapsed (`items[].keyword_info.search_volume`). A leaf is *used* when one of its values appears in something the agent wrote *after* receiving it and not before. Values are matched as word n-grams ($n \leq 4$): short strings as a whole, prose by any 4-gram, numbers in their plain and grouped forms. Values too generic to attribute (booleans, integers below 100, one-word strings shorter than four characters) are not *judgeable*. A value that already appears in the call's own arguments is an *echo*: not a use, but the record's key (the keyword next to its volume), and it is always kept.

Projection. For each tool, the *aggressive* projection keeps the used fields and the echoes; the *prudent* one also keeps every field that was never judgeable. Fields never seen while learning pass



Matching. A field is *used* when one of its values appears, as a word n-gram ($n \leq 4$), in what the agent writes *after* the call: later call arguments and the final answer. S2 also matches Unix times written as dates and minor units written as amounts.

Projection (prudent). Keep the fields used in any training run, the fields never judgeable, the identifying fields (S1: `id`, `*_id`, `url`, `name`...) and fields never seen; drop the rest. On a new run the dropped fields are listed in the `[trimmed]` line, and `expand` brings them back. A value the agent translates (*succeeded* → *successful*) is not matched: one of the failure mechanisms discussed in Section 7.

Figure 2: How usage is detected, on an illustrative billing call (values simplified). Blue: values matched in what the agent wrote after the call, directly or through the safeguard S2; orange: a value the agent used after translating it, which matching cannot see.

through, so that a field added to an API is not hidden silently. Projected outputs are re-serialised without ASCII escapes.

Leave-one-out evaluation. For each trace, the projection is learned from the other traces of the same task and applied to it. Two quantities are reported: the tool-output tokens left (counted with the `o200k_base` tokenizer, each output once), and the *misses*: calls in which the projection removes a field that this trace used later. Usage is a lower bound: a field read only while reasoning leaves no trace in the text, and reasoning is encrypted in the APIs used here.

Learning source. Usage can be learned from everything the agent wrote (*full*) or only from the arguments of later tool calls (*calls*), which is what a proxy between the agent and its tools can observe.

Structural compression. After projection, outputs can be cleaned (null and empty values removed) and written in a TOON-style encoding [16] implemented in the library (not the reference implementation): indented `key: value` lines, with arrays of uniform flat objects as tables whose field names appear once.

`expand`. At run time, a projected output is followed by a line listing the hidden fields, grouped two levels deep and largest groups first, and the agent can retrieve any of them with an `expand` tool. This turns a loss into a round trip, provided the agent knows what is missing.

Safeguards (version 0.2, Study 2). S1 keeps identifying fields (`id`, `*_id`, `doi`, `url`, `number`, `title`, `name` and similar) whatever the usage says. S2 recognises transformed values: Unix timestamps and ISO dates also match as written dates, in Italian and English; integers in money-named fields also match as major units with two decimals; URLs also match by their last segment.

4 Experimental design

Both studies follow protocols frozen before their runs and tagged in the repository (`protocol-v1`, amended as `protocol-v2`; Study 2 as `protocol-v3`). Every change made after freezing is listed, dated and explained in the protocol's "Deviations" section and summarised in Appendix A.

Cases. Five agents, all with `gpt-5.6-terra` at default parameters, in a loop of our own (OpenAI Responses API with function calling), because hosted tool integrations do not expose tool outputs to the client.

- **C1, editorial plan**: twelve tools of a search-data API (keyword research, trends, search results, rankings, page parsing) reached through its MCP server; the agent plans articles for the author's own website.
- **C2, content gaps**: search results and page parsing; the agent compares what ranks on Google with what the website has published.
- **C3, literature scan**: two tools on the OpenAlex catalogue (CC0), five results per search.
- **C4, billing support**†: seven tools of a simulated MCP server whose payloads follow the public OpenAPI description of the Stripe API; the agent answers five customer tickets with known correct answers.
- **C5, CI triage**†: six tools of a simulated MCP server following the GitHub REST API description; the agent explains the week's CI failures on `main` and links them to issues and pull requests, again with known correct answers.

†Simulated payloads are generated from the OpenAPI schemas with a seeded generator; because we design them, C4 and C5 are reported separately and never pooled with the real-API cases. Inputs name only the author's own domain; a gate in the tool wrapper rejects any other domain for page parsing, ranking targets and `site:` queries.

Study 1. *Phase A*: eight live runs per case, used for usage, leave-one-out projections, learning source and transfer between C1 and C2 (on the tools they share). *Phase B*: the third run of each case, chosen before looking at the data, is replayed up to the final answer, changing only the tool messages: raw, prudent, aggressive, and prudent plus structural compression; two runs per arm; projections learned from the other seven runs. Headroom (`headroom-ai` 0.39.1) is applied to the same conversations as a token-only baseline, configured to compress everything with no protection of recent messages and a target ratio of 0.2: the setting most favourable to it, so that the comparison does not depend on a cautious default. *Phase C*: live runs on C1 and C4 with the prudent projection and `expand`, against raw runs, three per arm.

Study 2. The Phase B conversations are replayed again with the safeguards on (arm `v02`, and `v02` plus structural compression) against the raw arm, five runs per arm.

Measures. *Gates* are binary: the answer parses, has the required structure, and every fact it attributes to a tool (volumes, URLs, identifiers, run and issue numbers, quoted error lines) matches the raw tool outputs, whatever the agent was shown. *Thermometers* are graded: editorial metrics for C1, coverage metrics for C2 and C3, and the number of correct diagnoses and links for C4 and C5, whose correct answers are known. Results are reported as medians with ranges; with two to five runs per arm no significance test is claimed. The pre-registered rule for quality (H2) holds an arm to the raw arm's range on every thermometer.

Case	Runs	Calls	Tool-output tokens	Used	Prudent	Aggressive	Misses	Misses (calls only)
Editorial plan	8	173	817,865	32%	−63%	−68%	2/166	69/166
Content gaps	8	136	527,542	84%	−12%	−16%	9/90	26/90
Literature scan	8	61	2,262,885	19%	−70%	−81%	3/61	61/61
Billing support†	8	84	154,833	17%	−69%	−84%	9/84	61/84
CI triage†	8	104	599,808	18%	−77%	−82%	3/64	64/64

Table 1: Phase A, eight live runs per case. *Used*: share of tool-output tokens left by the in-sample aggressive projection (used fields and record keys), a lower bound of use. *Prudent*, *Aggressive*: change in tool-output tokens on held-out runs. *Misses*: held-out calls in which the aggressive projection removes a field the run used later, learning from everything the agent wrote, or only from tool calls. †Simulated tools.

5 Study 1: what a learned projection removes, and what it breaks

5.1 How much the agent uses, and how much a projection removes

Table 1 summarises Phase A. Only a minority of the tool-output tokens carry fields the agent visibly used (with the record keys): 32% in the editorial plan, 19% in the literature scan, 17% and 18% in the two simulated cases. On runs it had not seen, the prudent projection removed 63%, 70%, 69% and 77% of the tool-output tokens of these cases, above the threshold of H1. The content-gap analyst is the boundary case: it cites most of what search results contain (URLs, titles, snippets) and its page parses are prose, which a field projection cannot touch; its projection removed 12%.

5.2 The learner must see the answer

Learning only from later tool calls, which is all a proxy between agent and tools can observe, multiplied the misses (H3): from 2/166 to 69/166 in the editorial plan, and to every trimmed call in the literature scan (61/61) and in CI triage (64/64): those agents use what tools return only in the final answer. A projection can be *applied* anywhere, a proxy included, but it has to be *learned* from traces that contain the answer.

5.3 Projections are task-specific

Learned on the editorial plan and applied to the content-gap analyst, on the one tool they share, the projection missed a used field in 67/90 calls, against 9/90 for the analyst's own projection (H4). What an agent needs from a tool depends on the task, not only on the tool.

5.4 Does the answer survive? (Phase B)

Replaying the same conversation with projected tool outputs (Table 2) removed 59%, 68%, 69% and 77% of the tool-message tokens with the prudent projection, against 28%, 17%, 33% and 31% for Headroom on the same conversations. H2, as frozen, was rejected in almost every case: with two runs per arm the raw arm's range is often a single value, so any difference counts, including improvements. Reading the answers rather than the rule, the losses are concentrated in three mechanisms, each traced to a specific field.

Transformed values. In billing support, the end of the subscription period arrives as a Unix timestamp and the agent writes it as a date. Usage detection does not recognise the rewritten value, so the field is judged unused and removed; every projected arm then answered the customer who asked until when the service would be available with a vague sentence instead of the date.

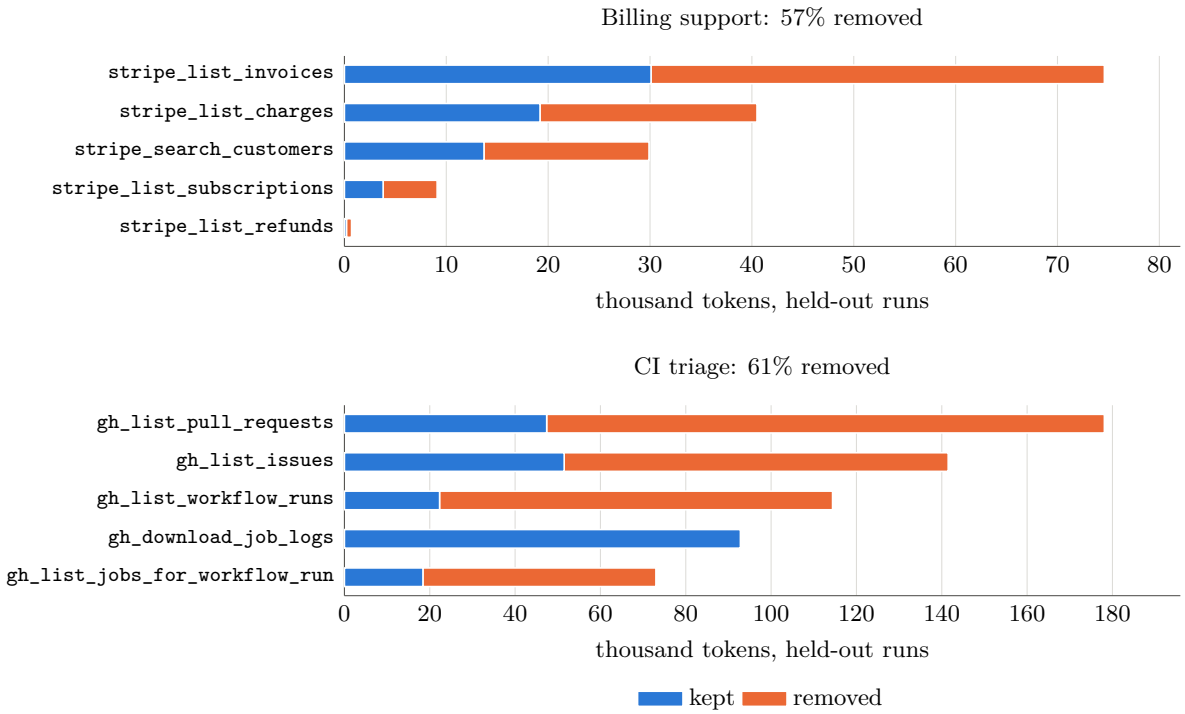


Figure 3: Where the tokens go, per tool, in the two cases with simulated tools (released in full): tool-output tokens on held-out runs, split into what the prudent projection keeps and what it removes. The projection here has the 0.2 safeguards on (Section 6), as the library now does by default, so it removes less than the 0.1 projection of Table 1. This is the breakdown the telemetry report shows for every agent; generated from the traces by `experiments/paper_figures.py`.

Fields used only to reason. In CI triage the projection removed issue titles: the agent reads them to decide which issue a failure belongs to, but writes only the issue number. Without titles, every projected arm failed to link the staging-deployment failure to the issue that reports it.

Invented identifiers. In the literature scan, the work identifier arrives as a URL and the agent writes its last segment, so the identifier field was removed. Asked to report it, the agent did not write `null` as instructed: it wrote sequential identifiers starting from the example in the system prompt. The DOIs it reported were correct; the identifiers did not exist. With the full outputs, in all eight live runs, every identifier was real. Removing an identifier is therefore not only a loss of information: it can cause the agent to fabricate one, the failure mode known for references and package names [17, 18].

5.5 Live loop with `expand` (Phase C)

In live runs with the prudent projection and `expand` (Table 3), billed input tokens fell by 56% in the editorial plan and 32% in billing support, uncached input tokens by 47% and 54%, and the cost per run by 40% and 25%. Every run passed every gate. The agents did use `expand`: a median of 1 call per run in the editorial plan and 6 in billing support, where the agent recovered the hidden period end in two of three runs. The raw arm itself misdiagnosed one ticket in two of its three runs, a reminder that run-to-run variation in these tasks is large compared with the effects measured.

Case	Arm	Tool tokens	Gates passed	Band rule
Editorial plan	raw	114,712	1/2	
	prudent	−59%	1/2	does not hold
	aggressive	−64%	2/2	does not hold
	structural	−60%	1/2	does not hold
	Headroom (tokens only)	−28%	–	–
Content gaps	raw	50,488	0/2	
	prudent	−12%	0/2	does not hold
	aggressive	−16%	0/2	does not hold
	structural	−13%	0/2	holds
	Headroom (tokens only)	−10%	–	–
Literature scan	raw	227,433	2/2	
	prudent	−68%	0/2	does not hold
	aggressive	−79%	0/2	does not hold
	structural	−72%	1/2	does not hold
	Headroom (tokens only)	−17%	–	–
Billing support†	raw	16,931	2/2	
	prudent	−69%	2/2	does not hold
	aggressive	−84%	2/2	does not hold
	structural	−73%	2/2	does not hold
	Headroom (tokens only)	−33%	–	–
CI triage†	raw	74,976	2/2	
	prudent	−77%	2/2	does not hold
	aggressive	−82%	2/2	does not hold
	structural	−78%	2/2	does not hold
	Headroom (tokens only)	−31%	–	–

Table 2: Phase B: replay of one run per case, only tool messages changed, two runs per arm. *Tool tokens*: raw size, then change per arm. *Gates*: runs in which every gate passed. *Band rule*: H2 as frozen.

6 Study 2: safeguards

Study 2 replayed the Phase B conversations again, 5 runs per arm, with the projection learned with both safeguards (arm `v02`) and with the same projection followed by structural compression (Table 4).

Pre-registered hypotheses. All four held as frozen. *H5*: in billing support the `v02` arm diagnosed all five tickets correctly in 5/5 runs, the period end included, where the 0.1 projection had lost it. *H6*: in the literature scan every `v02` run passed the gates (5/5), so no identifier was invented; with 0.1 the prudent arm had passed in 0/2 runs. *H7*: in CI triage every `v02` run found all four expected links, including the one lost in Phase B. *H8*: the `v02` arm still removed 53%, 46%, 57% and 61% of the tool-message tokens in the editorial plan, the literature scan, billing support and CI triage, against 28%, 17%, 33% and 31% for Headroom (Figure 5). The safeguards cost part of the saving (compare 68% with 46% in the literature scan) and bought back the three lost pieces of information.

A fourth mechanism, found after the fact. Reading the billing-support answers, we found that the frozen test for one ticket was too lenient: it accepted the diagnosis when the word "funds" appeared anywhere in the ticket, so advice such as "check that you have sufficient funds" passed it even when the diagnosis did not give the reason for the declined payment. With a strict test on the diagnosis alone (added after the data were seen, and reported beside the frozen result rather than instead of it), the raw arm gave the reason in 5/5 runs and the `v02` arm in 0/5. The reason

Case	Arm	Input tokens	Uncached	Turns	Expand calls	Gates
Editorial plan	raw	847,304	107,274	10	0	3/3
	prudent	374,748	57,117	10	1	3/3
Billing support†	raw	47,202	22,097	4	0	3/3
	prudent	32,161	10,239	5	6	3/3

Table 3: Phase C: live runs, raw tools against the prudent projection with `expand`; medians of three runs. Input tokens are billed tokens from the API, cache included.

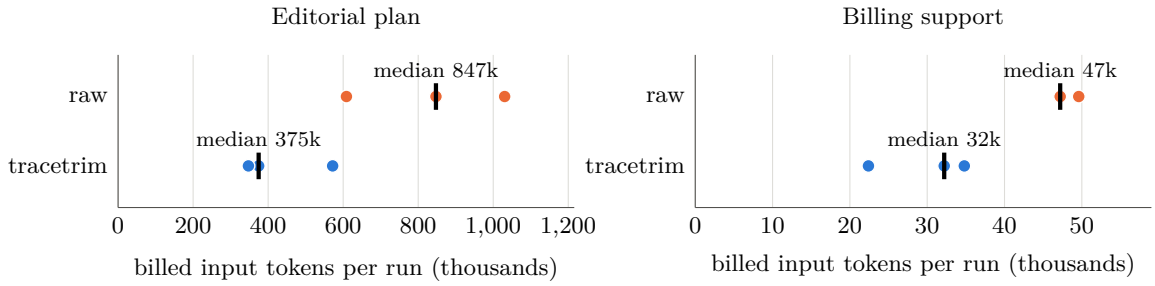


Figure 4: Phase C, every run: billed input tokens per run with raw tools (orange) and with the prudent projection and `expand` (blue); the bar marks the median. Three runs per arm: the spread is part of the result.

arrives as an English sentence and the agent writes it in Italian: a *translated* value, which neither usage detection nor safeguard S2 recognises. The `v02` answers were not wrong ("the card was declined") but less informative.

Format changes behaviour. Cleaning and TOON encoding remove no information, yet adding them to the `v02` projection changed answers. In the editorial plan the gates passed in 2/5 runs instead of 5/5, each time on the same keyword: the search-volume tool, which the prompt names as the source of all volumes, returned that keyword without a volume, while another tool gave one. With JSON the agent used the other tool's figure; with the denser format it wrote zero. In billing support the arm had all tickets right, under the frozen test, in 2/5 runs. The format did not hide data: it shifted how the model resolved an ambiguity. The evidence is thin (one keyword, five runs), and the encoder is the library's own minimal TOON-style implementation, not the reference one [16]; but it is a reminder that a format the model reads less often is not a neutral choice, and its saving has to be validated like any other change to what the agent reads.

The content-gap case. The content-gap analyst failed the gates in most runs of every arm, the raw one included (2/5): when the search API returns a redirect link instead of a result's URL, the agent reconstructs the URL from the breadcrumb, against the instructions. The safeguards neither caused nor fixed this, and the projection saves little on this task (9%).

7 Discussion

What the agent uses can be learned, but only from complete traces. Across five agents, most of what tools return is never visibly used, and a projection learned from a handful of runs of the same task removes most of it on a new run. The learner must see the final answer: an MCP gateway placed between agent and tools can *apply* a projection, but learning from what it sees alone misses most of what the agent uses. The practical consequence is an architecture: learn offline from full traces, review, apply anywhere. Version 0.3 of the library adds a second route, not evaluated in the studies reported here: the proxy offers the agent one more tool, through

Case	Arm	Tool tokens	Gates passed	Band rule
Editorial plan	raw	114,712	5/5	
	v02	−53%	5/5	does not hold
	v02-structural	−55%	2/5	does not hold
	Headroom (tokens only)	−28%	–	–
Content gaps	raw	50,488	2/5	
	v02	−9%	1/5	does not hold
	v02-structural	−10%	0/5	does not hold
	Headroom (tokens only)	−10%	–	–
Literature scan	raw	227,433	5/5	
	v02	−46%	5/5	holds
	v02-structural	−52%	5/5	holds
	Headroom (tokens only)	−17%	–	–
Billing support†	raw	16,931	5/5	
	v02	−57%	5/5	does not hold
	v02-structural	−62%	5/5	does not hold
	Headroom (tokens only)	−33%	–	–
CI triage†	raw	74,976	5/5	
	v02	−61%	5/5	holds
	v02-structural	−63%	5/5	holds
	Headroom (tokens only)	−31%	–	–

Table 4: Study 2: replay of the same runs as Phase B, 5 runs per arm, safeguards S1 and S2 on. Columns as in Table 2.

which the agent hands over its final answer, so that the gateway sees complete traces and can learn and apply the safe part of a projection on its own.

Where usage detection is blind, a projection fails. The losses we found have one cause: the projection removes what the detector cannot see. The agent transforms values (timestamps into dates, cents into euros, URLs into short identifiers, English into Italian), or reads a field only to decide something (an issue title to match a failure). Safeguards can close known blind spots, and S1 and S2 closed three of the four we found, but the list of possible transformations is open. The general lesson is that a learned projection is a *proposal*, whose blind spots are exactly the fields that matter in ways the text does not show.

Identifiers are special. The most serious failure was not a loss but a fabrication: deprived of an identifier it was asked to report, an agent invented plausible ones. Identifiers are cheap to keep and costly to lose, so a projection should never remove them, whatever the usage says.

The recovery path matters. With `expand`, a wrong cut costs one round trip instead of a wrong answer: in the live runs the agents asked for hidden fields when they needed them, and the end-to-end cost still fell. This only works when the agent knows what is hidden, which is why the hidden fields are listed, grouped, after every trimmed output.

How to use it. These results suggest a workflow rather than an automatic switch: record a few runs of a task with full outputs; generate the telemetry report, which is useful on its own and changes nothing; review the proposed configuration, a plain list of kept and dropped fields; apply it with identifier protection and `expand`; and keep recording, so that a field the agent starts needing shows up in the next report. The library implements each step, including an MCP proxy that applies a configuration in front of any server without changes to the agent.

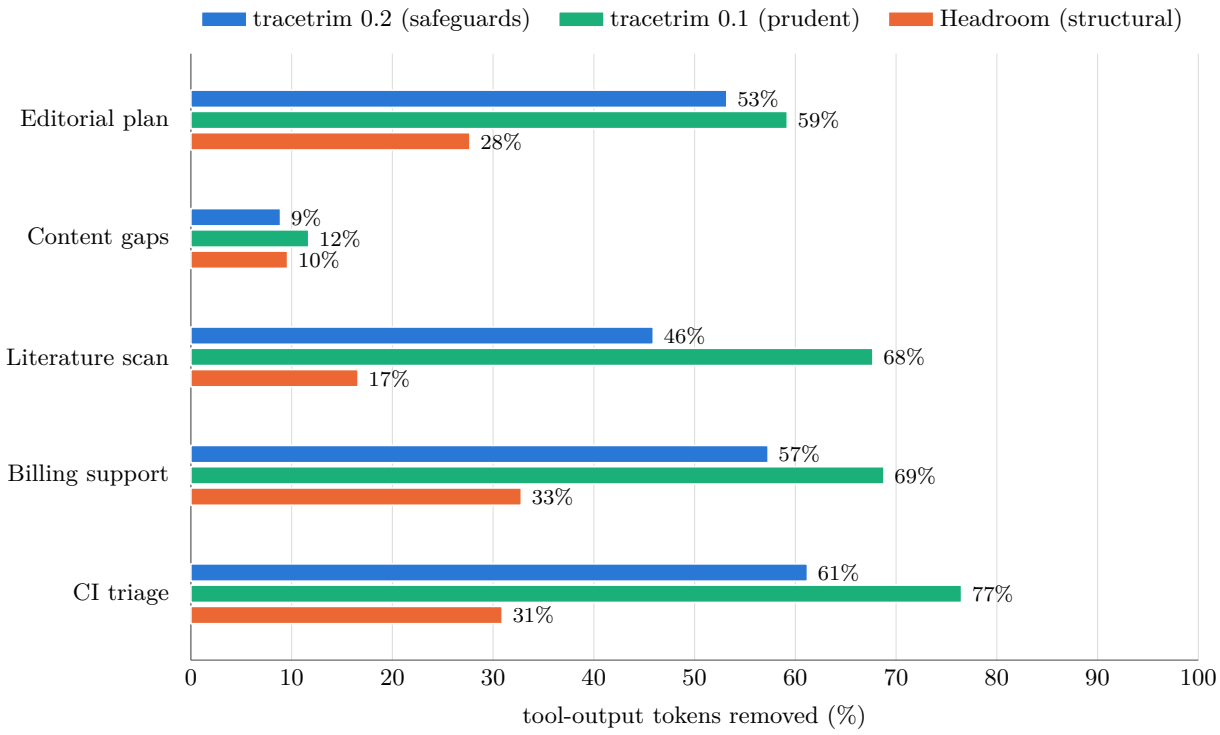


Figure 5: Tool-output tokens removed on the same conversations, per case: the learned projection without safeguards (0.1, Phase B), with them (0.2, Study 2), and Headroom 0.39.1 configured to compress everything. Token counts only: the effect on answers is in Tables 2 and 4.

8 Limitations

- **Small numbers of runs.** Eight live runs per case to learn, two to five replays per arm to judge quality. No significance test is claimed; differences inside the range of the raw arm are reported as not distinguishable from run-to-run variation, which in these tasks is large (in Phase C the raw arm itself misdiagnosed a ticket in two of three runs).
- **One model.** All runs used `gpt-5.6-terra` at default parameters. Whether other models use tool outputs differently, or read projected and TOON-encoded outputs differently, is untested.
- **Replay is not a live loop.** Phases B and Study 2 replay a fixed sequence of calls up to the final answer; in a live loop a projected output can also change which calls come next. Phase C measured the live loop on two cases only.
- **Simulated cases.** Two of the five cases use payloads we generated from public OpenAPI descriptions. Their shape is realistic, their content is not, and because we designed them they are reported separately.
- **Usage is a lower bound.** Reasoning is not observable in the APIs used, so fields used only while reasoning are invisible; this is the root of the failures reported, and safeguards can only cover the blind spots that are known.
- **Post hoc decisions.** Three measurement corrections were made after seeing data (deviations 8, 10 and 11); each is applied to every run, and the strict test of deviation 11 is reported beside the frozen one.

- **Third-party data.** Search results and parsed pages cannot be redistributed: for the two cases built on them, the released data are derived per-call records, enough to reproduce the usage analysis but not to replay the conversations.

9 Conclusion

Agents read far more than they use, and what they use can be learned from their own traces: on four of the five agents studied, a projection learned from other runs removed between half and three quarters of the tool-output tokens of a new run. Where it failed, it failed for a reason: the agent had transformed a value, used a field only to reason, or, deprived of an identifier, invented one. Two safeguards, keeping identifying fields and recognising transformed values, removed the losses that motivated them while keeping a saving between 46% and 61% on those agents; a fourth mechanism, translation, remains open. The method is therefore best used as it is implemented: as a proposal learned from complete traces, reviewed by a person, applied with identifiers protected and with a way back for the agent when a cut was wrong.

AI assistance and authorship

The author set the research question, the constraints of the study (own domains only, providers whose terms allow publication, the spend limit) and the requirement that the result be a usable tool, and approved each protocol before it was frozen. An AI coding agent (Claude Code, model Claude Opus), working in the author's project under the author's direction, proposed the protocols, wrote the library and the experiment code, ran the experiments and the analyses, and drafted this manuscript; a sub-agent compiled the bibliography, each entry verified against a primary source. The simulated payloads are generated by code from public OpenAPI descriptions, not written by a model. No human annotation was performed beyond the author's review. The author is responsible for the decision to publish and for the final text.

Funding and competing interests

This study received no external funding. API costs were paid by the author. The author has no financial or other relationship with OpenAI, DataForSEO, OpenAlex, Stripe, GitHub or the authors of Headroom and TOON.

Code and data availability

The library (`tracetrim`, MIT licence), the experiment code, the frozen protocols with all deviations, the aggregate results and the scripts that generate every number and table of this paper are released at `https://github.com/alepom/tracetrim` and archived on Zenodo with the dataset (DOI 10.5281/zenodo.23954125). Traces of the literature-scan case (CC0 catalogue) and of the two simulated cases are released in full. For the two cases built on the search-data API, whose terms do not license redistribution of results, only derived per-call records are released: tool, field paths used, echoed and judgeable, and token counts, which reproduce the usage analysis without the provider's data. The simulated servers and the payload generator are included, with the exact versions of the OpenAPI descriptions they follow.

References

[1] Atlassian Labs. mcp-compressor: An MCP server wrapper for reducing tokens consumed by MCP tools. https://github.com/atlassian-labs/mcp-compressor, 2026. Version 0.40.0 (latest release at access date). Apache-2.0. Accessed 2026-11-04.

[2] Headroom Contributors. Headroom: Compress tool outputs, logs, files, and RAG chunks before they reach the LLM. https://github.com/headroomlabs-ai/headroom, 2026. Python package headroom-ai, Apache-2.0. The version used for the measurements is stated in the text. Accessed 2026-11-04.

[3] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.825. URL https://aclanthology.org/2023.emnlp-main.825/.

[4] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.91. URL https://aclanthology.org/2024.acl-long.91/.

[5] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55321–55375, 2026. URL https://proceedings.mlr.press/v306/kang26b.html.

[6] Ádám Kovács. Squeez: Task-conditioned tool-output pruning for coding agents, 2026. URL https://arxiv.org/abs/2604.04979.

[7] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.391. URL https://aclanthology.org/2023.emnlp-main.391/.

[8] Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management, 2025. URL https://arxiv.org/abs/2508.21433. Presented at the 4th Deep Learning for Code (DL4C) Workshop at NeurIPS 2025.

[9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://aclanthology.org/2024.tacl-1.9/.

[10] lourencomaciel. Sift: Reliability gateway for AI tool output. https://github.com/lourencomaciel/sift-gateway, 2026. Python package sift-gateway, version 0.4.4, https://pypi.org/project/sift-gateway/. MIT. Accessed 2026-11-04.

[11] Model Context Protocol. Model context protocol specification, version 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28, 2026. Latest version as of access date; source at https://github.com/modelcontextprotocol/modelcontextprotocol. Accessed 2026-11-04.

[12] Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 19327–19352, 2023. doi: 10.52202/075280-0848. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/3d77c6dcc7f143aa2154e7f4d5e22d68-Abstract-Conference.html.

[13] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.57. URL https://aclanthology.org/2024.findings-acl.57/.

[14] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html.

[15] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 68539–68551, 2023. doi: 10.52202/075280-2997. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

[16] Johann Schopplich. TOON specification: Token-oriented object notation, version 4.4 (working draft). https://github.com/toon-format/spec, October 2026. Dated 2026-10-08. Reference implementation: https://github.com/toon-format/toon. Accessed 2026-11-04.

[17] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3687–3706. USENIX Association, 2025. ISBN 978-1-939133-52-6. URL https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen.

[18] William H. Walters and Esther Isabelle Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13(1):14045, 2023. doi: 10.1038/s41598-023-41032-5.

[19] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/bda88ed2892f5e61c9a9bf215c566913-Abstract-Conference.html.

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629.

A Deviations from the protocols

All are dated and explained in full in the repository's `protocol.md`; none was hidden or left unapplied.

0. A bug in the agent loop dropped the system prompt after the first turn; found in the pilot and fixed before Phase A. Pilot runs are excluded.
1. Prompts were written during the pilot rather than at freezing, and fixed before Phase A.
2. Tool-level caps made explicit: at most five results per literature search; a safety guard at 80 calls.
3. The literature catalogue was used without an API key, within its free daily allowance.
4. A verification pilot after the fix; one correction to the CI-triage gate (several job names per cause).
5. Simulated tools fixed before Phase A, with the OpenAPI versions and the presence rule documented.
6. Transport errors of the search-data server: retries reconnect; after three failures the agent receives an error and the run continues, flagged. Two editorial-plan runs are flagged.
7. The list of hidden fields shown with `expand` was grouped, largest groups first, before Phase C.
8. Post hoc: the literature-scan gate now requires a declared identifier to exist (it had accepted a valid DOI next to an invented identifier).
9. Phase C was delayed by a faulty resume script; no run was altered.
10. Post hoc: the CI-triage gate also splits job names on "/".
11. Post hoc: a strict test for one billing ticket, reported beside the frozen one.
12. Traces of the search-data cases are released as derived per-call records instead of pseudonymised traces, which could not have preserved the analysis.